

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures

COURSE CODE:CSA03 16

COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
for(int i = 0; i <= n; i++) {  
  
if(arr[i] == key)  
  
return i;  
  
}  
  
return -1;  
  
}
```

Debugged code :

main.c	Output
<pre>1 #include <stdio.h> 2 3 int linearSearch(int arr[], int n, int key) 4 { 5 for(int i = 0; i < n; i++) // Fixed loop condition 6 { 7 if(arr[i] == key) 8 { 9 return i; // Return index if key is found 10 } 11 } 12 return -1; // Return -1 if key is not found 13 } 14 15 int main() 16 { 17 int arr[] = {10, 20, 30, 40, 50}; 18 int n = sizeof(arr) / sizeof(arr[0]); 19 int key = 30;</pre>	<pre>Element found at index 2 === Code Execution Successful ===</pre>

2. Buggy Code in an array

```
#include <stdio.h>

int main() {

    int arr[5] = {10,20,30,40,50};

    for(int i=0;i<=5;i++)

        printf("%d ",arr[i]);

    return 0;

}
```

Expected output:

10 20 30 40 50

Debugged Code :

Programiz C Online Compiler

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int arr[5] = {10,20,30,40,50}; 4 for(int i=0;i<=5;i++) 5 printf("%d ",arr[i]); 6 return 0; 7 } 8</pre>		<pre>10 20 30 40 50 0 === Code Execution Successful ===</pre>

3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)

{

    int low = 0, high = n - 1;

    while(low < high)

    { int mid = (low + high) / 2;

      if(arr[mid] == key) return mid;
```

```
else if(arr[mid] < key)
```

```
low = mid;
```

```
else high = mid;
```

```
}
```

```
return -1;
```

```
}
```

Debugged Code :

The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program for binary search. The output panel on the right shows the result of the execution.

```
main.c
1  #include <stdio.h>
2
3  int binarySearch(int arr[], int n, int key)
4  {
5      int low = 0, high = n - 1;
6
7      while (low <= high)
8      {
9          int mid = (low + high) / 2;
10
11         if (arr[mid] == key)
12             return mid;
13         else if (arr[mid] < key)
14             low = mid + 1;
15         else
16             high = mid - 1;
17     }
18
19     return -1;
```

Output

Element found at index 3

=== Code Execution Successful ===

4. Buggy Code in Stack Push Operation

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int data) {
```

```
    if (top == MAX - 1)
```

```
        printf("Stack Overflow");
```

```
    else
```

```

        stack[top++] = data;
    }

int main() {
    push(10);
    push(20);

    printf("%d", stack[top]);
}

```

Debugged Code :

The screenshot shows the Programiz C Online Compiler interface. The code editor contains the following C program:

```

main.c
5  int top = -1;
6
7  void push(int data)
8  {
9      if (top == MAX - 1)
10         printf("Stack Overflow\n");
11     else
12         stack[++top] = data;    // Fixed
13 }
14
15 int main()
16 {
17     push(10);
18     push(20);
19
20     printf("Top element = %d\n", stack[top]);
21
22     return 0;
23 }

```

The output panel on the right shows the following results:

```

Top element = 20

=== Code Execution Successful ===

```

5. Buggy Code in Stack POP Operation

```

#include <stdio.h>

#define MAX 5

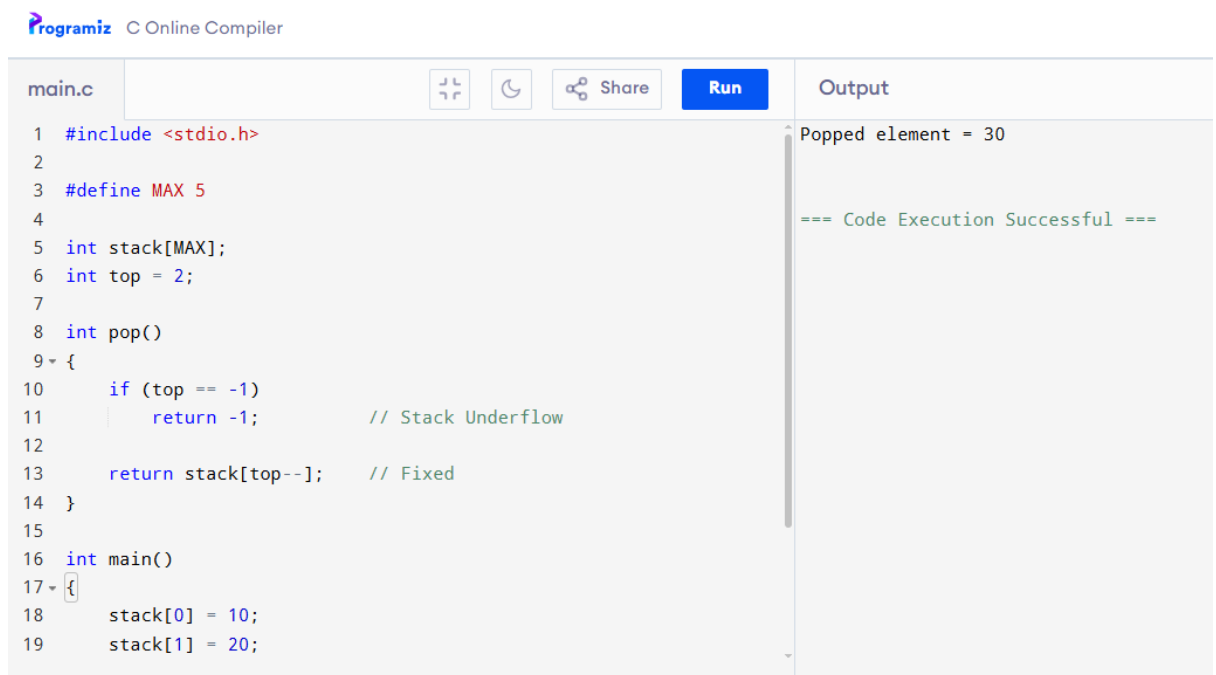
int stack[MAX];

int top = 2;

int pop() {
    if (top == -1)
        return -1;
    return stack[++top];
}

```

Debugged Code :



The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` is as follows:

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = 2;
7
8 int pop()
9 {
10     if (top == -1)
11         return -1; // Stack Underflow
12
13     return stack[top--]; // Fixed
14 }
15
16 int main()
17 {
18     stack[0] = 10;
19     stack[1] = 20;
```

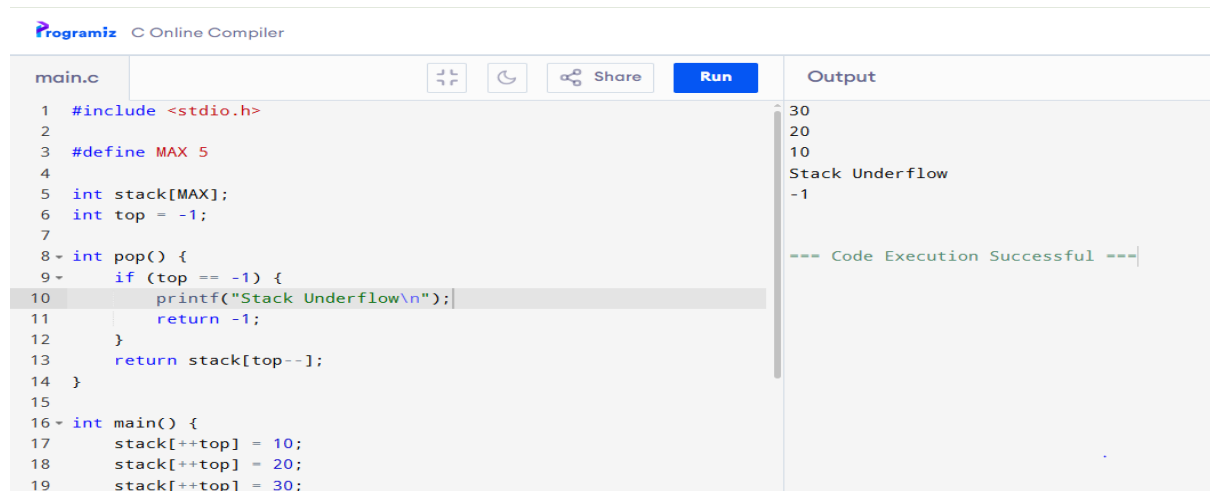
The output on the right shows:

```
Popped element = 30
=== Code Execution Successful ===
```

6. Buggy Code in underflow operation

```
int pop() {
    if(top == 0) {
        printf("Underflow");
        return -1;
    }
    return stack[top--];
}
```

Debugged Code :



The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` is as follows:

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 int pop() {
9     if (top == -1) {
10         printf("Stack Underflow\n");
11         return -1;
12     }
13     return stack[top--];
14 }
15
16 int main() {
17     stack[++top] = 10;
18     stack[++top] = 20;
19     stack[++top] = 30;
```

The output on the right shows:

```
30
20
10
Stack Underflow
-1
=== Code Execution Successful ===
```

7. Find the issue in the following code snippet.

```
#include<stdio.h>

#define MAX 5

void main()
{
    int queue[MAX],x;
    int front = 0, rear = 0;
    printf("Enter the value");
    scanf("%d",&x);
    void enqueue(int x) {
        if(rear == MAX) {
            printf("Overflow");
            return;
        }
        else{
            queue[rear++] = x;
            printf("Value is %d", queue[rear]);
        }
    }
}
```

Debugged Code :



The screenshot shows the Programiz Online Compiler interface. The code editor on the left contains the following C code:

```
main.c
1  #include <stdio.h>
2
3  #define MAX 5
4
5  int queue[MAX];
6  int front = 0, rear = 0;
7
8  void enqueue(int x)
9  {
10     if (rear == MAX)
11     {
12         printf("Queue Overflow\n");
13         return;
14     }
15
16     queue[rear++] = x;
17     printf("Value inserted: %d\n", x);
18 }
19
```

The output panel on the right shows the following text:

```
Enter the value: 25
Value inserted: 25

=== Code Execution Successful ===
```

8. Buggy code in node creation

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *newNode;
```

```

newNode->data = 10;
newNode->next = NULL;

printf("%d", newNode->data);

return 0;
}

```

Debugged Code :

The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` defines a `Node` struct with `data` and `next` pointers. In the `main` function, a new node is allocated using `malloc`, its `data` field is set to 10, and its `next` pointer is set to `NULL`. The `data` field is then printed using `printf`. The output shows the number 10 and a success message.

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *newNode;
11
12     // Allocate memory for the new node
13     newNode = (struct Node *)malloc(sizeof(struct Node));
14
15     if (newNode == NULL) {
16         printf("Memory allocation failed.\n");
17         return 1;
18     }
19
20     newNode->data = 10;
21     newNode->next = NULL;
22
23     printf("%d", newNode->data);
24
25     return 0;
26 }

```

Output: 10
=== Code Execution Successful ===

9. Include missing statements and display the list of elements in linkedlist

```

void display(struct Node *head)
{
while(head != NULL);
{
printf("%d ", head->data);
head = head->next;
}}

```

Debugged Code :

The screenshot shows the Programiz C Online Compiler interface. The code defines a `Node` struct and a `display` function. The `display` function uses a `while` loop to traverse the linked list, printing each node's `data` field followed by a space. The `main` function (partially visible) would create a linked list with three nodes containing values 10, 20, and 30. The output shows the linked list: 10 20 30 and a success message.

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Define the structure
5 struct Node
6 {
7     int data;
8     struct Node *next;
9 };
10
11 // Function to display the linked list
12 void display(struct Node *head)
13 {
14     while (head != NULL)
15     {
16         printf("%d ", head->data);
17         head = head->next;
18     }
19 }

```

Output: Linked List: 10 20 30
=== Code Execution Successful ===

10. Bug in Queue Full Condition-Identify and fix the errors.

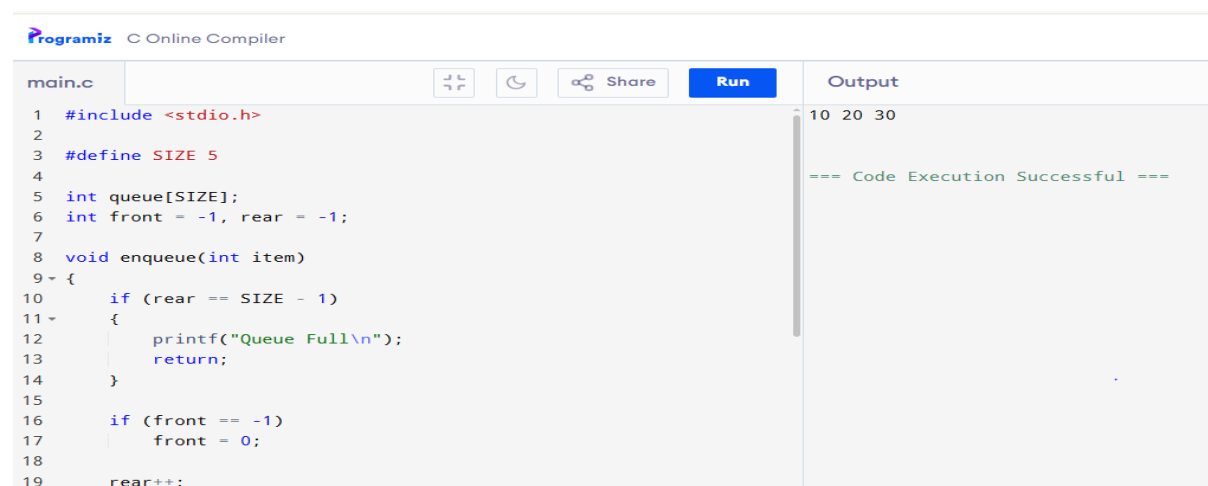
```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
    if(rear == SIZE-1)
    {
        printf("Queue Full");
        return;
    }

    if(front == -1)
        front = 0;

    rear = (rear + 1) % SIZE;
    queue[rear] = item;
}
```

Debugged Code :



The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains the following C code:

```
main.c
1 #include <stdio.h>
2
3 #define SIZE 5
4
5 int queue[SIZE];
6 int front = -1, rear = -1;
7
8 void enqueue(int item)
9 {
10     if (rear == SIZE - 1)
11     {
12         printf("Queue Full\n");
13         return;
14     }
15
16     if (front == -1)
17         front = 0;
18
19     rear++;
```

The output panel on the right shows the execution results:

```
10 20 30
=== Code Execution Successful ===
```

Code of Conduct:

I _G.Hima Bindu(192511377)_certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.HimaBindu

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation